Verification Continuum

HAPS® ProtoCompiler hRun User Guide

Version V-2025.06-3, April 2026

SYNOPSYS®

Copyright and Proprietary Information Notice

© 2026 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys, Ansys, the Synopsys and Ansys logos, and other Synopsys trademarks are available at https://www.synopsys.com/company/legal/trademarks-brands.html.
Other company or product names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Feedback

Contents

1. **Preface** 6
   Related Documentation 6

2. **Introduction** 8
   Benefits 8

3. **Debugging Workflow with hRun** 10
   Starting the hRun Tool 11

4. **Managing Hardware Configuration** 13
   `setup configure` 13
   `setup info` 15
   `setup control` 16
   `setup clear` 19
   `setup disconnect` / `setup connect` 19
   `setup attach` 22
   Exiting a hRun Session 22

5. **Managing hRun Jobs** 24
   `host hrun_scan` 24
   `host hrun_control` 24
   Terminating hRun Jobs 25

6. **The Design Prototype** 26
   Forcing Signal Values in the Design 26
   `signal force` 26
   `signal getvalue` 27
   `signal release` 28

`signal list` . . . 28
Managing Memory in the Design . . . 29
`memory list` . . . 30
`memory write` . . . 30
`memory read` . . . 30

7. Debugging the Design Using hRun . . . 32
Configuring the IICE Sample Clock . . . 32
Accessing IICE Information . . . 33
Configuring the IICE Sampler . . . 35
Starting and Stopping IICE Runs . . . 36
Defining IICE Trigger Settings . . . 37
IICE Trigger Commands . . . 38
CEL Syntax . . . 38
CEL Trigger Examples . . . 44
Setting the Target for Sample Data . . . 47
`dump` . . . 47
Running Deep Trace Debug . . . 48
Using BRAM for Debugging . . . 49
Using Global State Visibility for Debugging . . . 49
Cross Triggering . . . 52
Fast Track Debug . . . 53
Debug ECO . . . 54

8. Appendix . . . 56
Hardware Mapping File (HMF) Format . . . 56
Design Configuration File (DCF) Format . . . 56
Key Features of the DCF File . . . 57
Configuring Global Options in hRun . . . 59
Memory and Signal File Formats . . . 60

1

Preface

This section contains the following topics:

- Related Documentation

About this Guide

The *HAPS ProtoCompiler hRun User Guide* describes the functionality of the hRun tool and how to use its features.

This document is intended for users who are familiar with the HAPS ProtoCompiler product range.

Contents of this Guide

The *HAPS ProtoCompiler hRun User Guide* have the following chapters:

- Introduction
- Debugging Workflow with hRun
- Managing Hardware Configuration
- Managing hRun Jobs
- The Design Prototype
- Debugging the Design Using hRun
- Appendix

Related Documentation

The following product documents are included within the HAPS Prototyping tools. Unless otherwise specified, all documents can be accessed through the help system and as PDF documents:

- *HAPS® Prototyping User Guide*
- *HAPS® Prototyping Reference Manual*
- *HAPS® Prototyping Command Reference Manual*

- *HAPS® Prototyping Compiler and Mapper Guide*
- *HAPS® Prototyping Debugging Environment Reference Manual* (HAPS-100 only)
- *HAPS® Prototyping Debugger User Guide* (HAPS-100 only)
- *HAPS® Prototyping Instrumentor User Guide*
- *HAPS® Prototyping Message Reference Manual*
- *HAPS® Prototyping hRun User Guide* (HAPS-200 only)
- *HAPS® Prototyping Limited Customer Availability (LCA) Features Guide*
- *System Configuration Software Handbook* (HAPS-100 only)
- Free and Open-Source Software (FOSS) information is in PDF format and is located in the install_dir/doc directory.

Additional documentation is available outside the software:

- HAPS hardware documentation is available on https://spdocs.synopsys.com/dow_retrieve/latest/home_public/haps.html.

2

Introduction

The Synopsys HAPS hRun software is a runtime tool that enables you to run, control, and debug designs on HAPS hardware using a unified Tcl shell interface. It supports runtime tasks, such as configuration, Global Signal Visibility (GSV), Deep Trace Debugging (DTD), dynamic force, and memory backdoor.

This guide introduces the Synopsys HAPS hRun, a Linux shell-based tool for configuring on HAPS-200 and HAPS-100 hardware platforms. This tool simplifies runtime operations by providing a unified, scriptable interface for design execution. hRun is required for HAPS-200 and optional for HAPS-100, but it is not supported for earlier systems.

To use this tool, first run the HAPS ProtoCompiler tool and export the runtime files from the HAPS ProtoCompiler. The HAPS hRun tool uses these files to configure the designs on the HAPS hardware.

Benefits

- **Unified Configuration and Debugging:** Combines configuration and debugging tasks within a single shell to streamline operations and reduce complexity.
- **Automated Hardware Management:** Automates hardware access, configuration, and license management, minimizing manual intervention.
- **Enhanced Integration:** Coordinates with Synopsys HAV Scheduler (SHS), HAPS Resource Manager (HRM), and HAPS Utility (hUtils) and supports end-to-end hardware management.
- **Flexible and Modular Flows:** Supports full automation, user customization through Design Constraint File (DCF), and modular extensibility.
- **Improved Debugging**: Allows to attach a new hRun session to run designs for live debugging within the same shell.
- **Scriptable Interface:** Provides Tcl scripting capabilities for automation and batch operations.

- **Improved Usability:** Offers improved shell features, such as command history, tab completion, and error highlighting.
- **Platform Flexibility:** Compatible with HAPS-200, HAPS-100, and mixed environments.

For more information, See the HAPS Utility documentation.

3

Debugging Workflow with hRun

Do the following to prepare to use hRun:

1. Prepare input files for hRun.

   Export the runtime files from the HAPS ProtoCompiler tool after place-and-route (P&R) is complete. Use this design database as the primary input for hRun, together with any additional required information.

   - Export runtime directory: Contains the compiled design output.
   - HMF (Hardware Mapping File): Maps hardware modules and FPGAs.
   - DCF (optional): Used for runtime customization.

2. Ensure that the HAPS hardware specified in the HMF file is:
   - available with appropriate cabling for this design, and
   - accessible from the host machine where hRun is executed.
3. Start the hRun shell to begin the session. See Starting the hRun Tool.
4. Configure the design and start design clocks. See setup configure.
5. If required, override values on pre-instrumented signals in the design. See Forcing Signal Values in the Design.
6. If required, perform memory management. See Managing Memory in the Design.

   The prototype for the DUT (Design Under Test) is now fully operational. This is the primary phase for core validation and debugging tasks.

   Note:

   hRun enables this environment but does not directly participate in the design validation.

7. When the DUT is no longer needed, remove the DUT and release hardware resources. See setup clear.

Starting the hRun Tool

- Execute the following command to start the hRun tool in interactive mode.

  ```
  hRun -tcl <file> -shell <args> [-log <file>]
  ```

- Execute the following command to start the hRun tool in batch mode.

  ```
  hRun -tcl <file> -batch <args> [-log <file>]
  ```

- `-tcl <file>`: Specify the Tcl script file to run on startup.

Example 1 hRun.tcl script

```
set hmf_file [lindex $argv 0]
set design_dir [lindex $argv 1]

setup configure -designdir $design_dir -hmf $hmf_file -workdir
 hrun_dir
memory write dut_inst.RAM[127:0] -radix binary DF_write_bin.txt
memory read dut_inst.RAM[127:0] -radix binary DF_write_bin_out.txt
 -size 10
setup clear
exit
```

- `<args>`: Specify the arguments to bypass in the Tcl script file.

  In the sample *hRun.tcl* script, argument variables have been specified for *$hmf_file* and *$design_dir*. You can specify the value of these arguments in the command as options. For example:

  ```
  hRun -batch -tcl hRun.tcl <hmf_file_path> <design_dir_path>
  ```

  In this example, <*hmf_file_path*> and <*design_dir_path*> are arguments (values) that will be used in place of the variables in the script (*$design_dir* and *$hmf_file*) during script execution.

- `[-log <file>]`: Specify the standard out/error log filename.

To display help information use `help` command.

```
---------- Start command 'help' ----------
dump
exit
help
host
iice
memory
module
option
program_version
setup
```

Feedback

```
signal
```

4

Managing Hardware Configuration

Use the following commands to automate hardware management, improve debugging workflows, and ensure optimal use of licenses and resources for your Design Under Test (DUT).

Command	Usage
setup configure	• Programs the hardware • Configures the FPGA • Starts the DUT
setup clear	Removes the DUT image from the hardware and releases associated hardware resources.
setup control	Controls the HAPS hardware resources of this setup.
setup attach	Attaches a process to an active session to debug RTL.
setup disconnect / setup connect	Terminates the current hRun session and restarts it in a new process without disrupting the DUT running on hardware.
setup info	Retrieves key configuration information saved in the tool memory.

setup configure

Execute the `setup configure` command to:

- Download the design on the hardware.
- Program the HAPS hardware identified by the HMF and as defined in the TSD file, including connector voltages, PLL settings, and GCLK routing.
- Configure the FPGAs using the downloaded bitstream files (PDI) from the design directory.
- Check out one runtime license token per FPGA and any licenses required by used memory models.
- Start the DUT clocks.

1. Prepare the export runtime directory.

   Ensure that the export runtime directory contains the output generated by the `export runtime` command from ProtoCompiler after place-and-route (P&R) is complete. This directory serves as the input to hRun and is not modified during runtime. All files generated during the session are stored in the specified working directory (`workdir`).

   Note:

   The `export runtime` command is executed multiple times during the compile flow. However, only the final export, which contains the complete bit files (also called PDI) and database files (also called RTDB), is valid for use with hRun.

2. Find an available setup compatible with the compiled design. This setup is defined by a HMF file that is used in the next step.

3. Run the `setup configure` command to configure the hardware. This command expects the required resources to be available and accessible from the host computer running this command.

   ```
   setup configure -designdir <design_directory> -hmf <hmf_filepath>
   -workdir <workdir_path> [-dcf <dcf_filepath>] -overwrite_fpga <mode>
   ```

   - `-designdir <design_directory>`: Path to the exported runtime directory from ProtoComplier.
   - `-hmf <hmf_filepath>`: Path to the Hardware Mapping File (HMF), which defines the hardware modules and FPGAs to be used..
   - `-workdir <workdir_path>`: Path to an empty working directory for this configuration session. It stores all files generated during this run and acts as a proxy for the current session. It is used to identify the session in commands such as `setup connect`. Make a note of the path to the working directory for later use.

     A new and empty work directory is recommended for each execution of the command.
   - `-dcf <dcf_filepath>`: (Optional) Path to a Design Configuration File (DCF) for customizing runtime behavior.
   - The DUT is now configured and ready for use.
   - Key configuration details are stored in memory and can be retrieved using the `setup info` command.
   - Runtime licenses remain checked out until the design is explicitly cleared using `setup clear`.

Feedback

setup info

The `setup info` command retrieves static configuration information associated with a previously configured setup without accessing the hardware. It is useful for querying setup-specific details, such as configuration identifiers, FPGA mappings, and file paths.

```
setup info -key <keyword>
```

where `<keyword>` specifies the information to retrieve.

Table 1 Supported Keywords and Return Formats

Keyword	Description	Return format
`config_id`	Returns the HANDLE ID of the selected user FPGA. This ID may be used in UMRBus3 designs to construct MCAPIM names.	TCL str
`fpga_id`	Returns the FPGA ID of the selected user FPGA.	TCL str
`occupied_resources`	Lists the FPGAs used in the setup (useful in Multi-Design Mode).	TCL Dict
`dcf_path`	Provides the path to the DCF file.	TCL str
`workdir`	Provides the path to the working directory that is set in the `setup configure` command.	TCL str
`designdir`	Provides the path to the design directory that is set in the `setup configure` command.	TCL str
`current_scope`	Provides the scope name of the current design.	TCL str
`current_state`	Provides the current state of the hRun process. The state includes “None”, “Configured”, “Connected”, “Disconnected”, “Cleared”, “Error”.	TCL str

Feedback

Table 2 Current State of hRun Process

State	Description
Configured	The design has been successfully downloaded onto the HAPS hardware. This state is entered after a successful execution of the `setup configure` command. The HAPS system is now programmed and ready for use.
Connected	A hRun process has successfully reconnected to an already running design on HAPS. This state is entered after a successful `setup connect` command. The connected process has full control over the running design. This requires a previously configured setup that was earlier disconnected using setup disconnect.
Disconnected	The hRun process has disconnected from the HAPS setup without stopping the running design. This state is entered after a successful `setup disconnect` command. A background process continues to manage the setup, allowing the current hRun session to disconnect safely. Another hRun instance can later reconnect to the running design using `setup connect`.
Cleared	The setup has been terminated and all resources have been released. This state is entered after a successful `setup clear` command. Both the FPGA configuration and HAPS system resources are cleared and released. The setup handle is closed and cannot be reused.
Error	An error occurred during the execution of a `setup` command. This state indicates that the previous command did not complete successfully. Check the error messages in the log for details on the failure and take corrective action before proceeding.

setup control

The `setup control` command controls the configured HAPS hardware setup. It queries hardware status and perform control actions. This command requires exclusive access to all hardware modules in the design.

Note:

This command is not supported in secondary sessions initiated via `setup attach`. It scans all HAPS systems visible to the host and opens only those modules used in the current setup.

```
setup control [-action <action_name>] [-fpga <value>] [-get_status
<value>] [-port <value>] [-value <value>] -expert_action <action_name>
```

Feedback

- `-action <action_name>`: Specifies the control action to perform. The *<action_name>* types include:
  - `usr_local_reset`: Controls the `usr_local_reset` signal for one or more FPGAs. The `-value` parameter determines the logical value of the reset signal.

    ```
    setup control -action usr_local_reset -fpga {<fpga1> <fpgan>} -value
    <value>
    ```

  - `export_infraip_train_report`: Generates and exports the Infrastructure IP training reports (e.g., HSTDM, MGTDM) captured during FPGA configuration. This report assists in debugging hardware bring-up issues.

    ```
    setup control -action export_infraip_train_report
    ```

  - `start_fpga_jtag`: There is no dedicated JTAG connector on the HAPS module. Instead the built-in UMRBus offers JTAG communication for your application. The virtual cable (XVC) from Vivado Hardware Manager is a TCP/IP-based protocol that acts like a JTAG cable. This command launches a virtual JTAG server for the specified FPGA. It enables a connection to the Vivado Hardware Manager for debugging or configuration.

    ```
    setup control -action start_fpga_jtag -fpga <value>
    ```

    Connect to JTAG using the Vivado Hardware Manager.

    ```
    connect_hw_server

    open_hw_target -xvc_url localhost:65137
    ```

    Note:
    This virtual JTAG is separate from any JTAG interface or transactor present in the DUT or RTL design.

  - `stop_fpga_jtag`: Terminates the virtual JTAG server for the specified FPGA. If multiple FPGAs are using the same JTAG server, stopping one FPGA will stop all associated FPGAs.

    ```
    setup control -action stop_fpga_jtag -fpga <value>
    ```

  Note:

  Training reports reflect the state at the time of `setup configure`, not the current runtime state.

- `-fpga <value>`: Specifies the target FPGA(s) for the action. Multiple FPGAs can be listed.

Feedback

- `-get_status <value>`: Retrieves real-time status information from the hardware. Supported status values include:

Value	Meaning	Return format
`config_id`	Returns the handle ID used in UMRBus3 designs.	Tcl string or dict
`user_capims`	Lists all MCAPIMs directly instantiated by the user.	Tcl string or dict
`hstdm_erd`	Returns the current ERD state.	Tcl string or dict
`train_status`	Returns the results of training of infra IP such as HSTDM.	Tcl string

- `-port <value>`: Specifies the port to use when starting the virtual JTAG server.
- `-value <value>`: Provides a value for the specified action (`0` to assert reset, `1` to de-assert).
- `-expert_action <action_name>`: Specifies the control action to perform. The *<action_name>* types include:
  - `list_all_pll_freq`: Prints clock frequencies for all configured GCLK PLLs on the HAPS modules.
  - `validate_design_signature`: When the design is running, any change to the FPGA images causes an error. The design session is terminated. It needs to be restarted using the `setup configure` command.

    This occurs when the HAPS hardware is power-cycled or the FPGA image is modified or cleared. Specify the `validate_design_signature` option to check FPGA images used in a valid setup and detect changes in the images. The command returns:
    - *TCL_OK* if there is no change in the FPGA images.
    - *TCL_ERROR* if there is a change in the FPGA images. It also lists the FPGAs with modified images. The original design is cleared, but the FPGAs with new images are not cleared as they are part of a working design.

setup clear

The `setup clear` command is used to release all runtime licenses, clear the FPGA configuration, remove hardware programming, and release all associated resources. The hardware status is updated in the resource manager, If HRM is enabled.

This is the final step in the runtime flow and marks the end of the DUT prototype lifecycle.

After this command is executed:

- The DUT prototype is permanently cleared from hardware.
- The hardware modules return to an idle state and are available for another user to download a different design.

Note:

To keep the DUT prototype running, use `setup disconnect`.

setup disconnect / setup connect

The `setup disconnect` and `setup connect` commands in hRun allow you to pause a current hRun session and resume it later, enabling flexible control of HAPS hardware across multiple scripts or processes. This is especially useful for managing long or modular test flows.

Single-Process Control

Only one hRun process can control a configured setup at any time. To transfer control to another process:

- The current process must first run `setup disconnect`.
- The new process can then use `setup connect` to resume control.

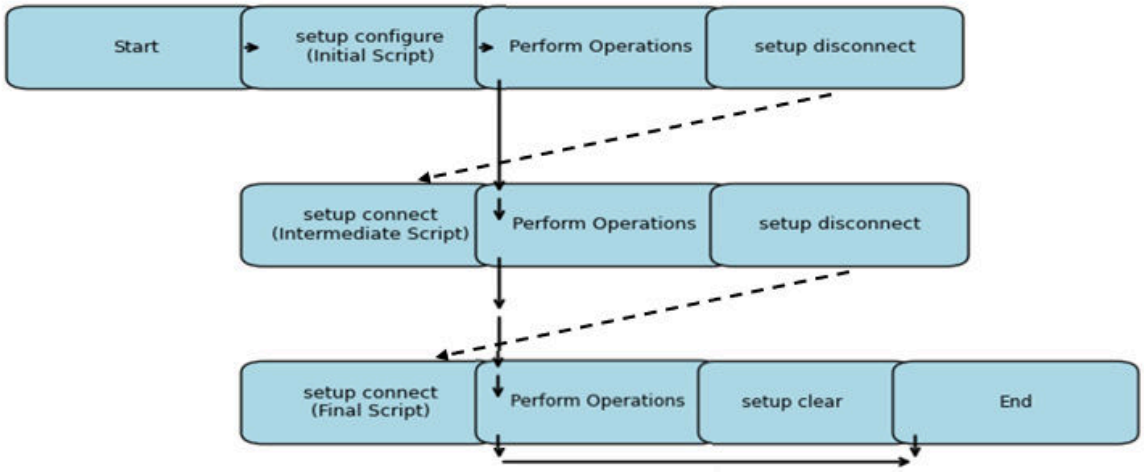
The setup and DUT (Design Under Test) can remain active and running even when no process is actively connected to it.

Use Case: Multi-Script Design Flow

This mechanism is commonly used to divide a test flow into multiple scripts. For example, one script to configure the setup and another to clear it after execution.

- First Script:
  - Use `setup configure` to program the hardware and start the DUT.
  - Ends the script with `setup disconnect` to keep the hardware running and while release control.

- Intermediate Scripts:
  - Start with `setup connect` to connect to the running hardware session.
  - Perform required operations such as debugging or accessing signals and memory.
  - End with `setup disconnect` to again release control while keeping the design active.
- Final Script:
  - Starts with `setup connect` to resume the session.
  - Ends with `setup clear` to reset the hardware and release all resources.



Note:

Typically, `setup disconnect` or `setup clear` is the final command in a Tcl script.

- Connect a New Session:

```
setup connect -workdir <workdir_path> -expert_action
force_disconnect_other_process
```

`-workdir <workdir_path>`: : Creates a separate hRun working directory called *workdir* for every design session.

`-expert_action force_disconnect_other_process`: Forces the connection by disconnecting another process, even if the other process is already connected. A maximum of one process can be connected at any time. Without this option, an error occurs if another process is already connected.

This command establishes a connection to an active design session on HAPS hardware using the specified working directory. This command is used to resume control of a previously configured setup that was disconnected using the `setup disconnect` command. Only one hRun process is connected to a setup at any given time.

- Checks that the hardware is still running, and the design is active.

- Disconnect the Current Session:

  ```
  setup disconnect
  ```

  This command transfers hardware control to a background process. When the next script is ready, use the `setup connect` command to reconnect to the running hardware.

  - A hRun process remains active in the background to maintain the hardware and license state.
  - Do not manually terminate this background process, as this may cause the design and hardware to enter an undefined state. To terminate a background hRun job, use the `host` commands to release the hardware.

End-of-Script Guidelines:

At the end of each script, choose one of the following:

- **setup clear**: Ends the design session, resets the hardware, and releases all resources.
- **setup disconnect**: Informs hRun to keep the design running while ending the current script. You can reconnect and resume operations as needed.

Note:

Create a separate hRun working directory for each design or for each run that uses different configuration settings. The `workdir` path is necessary to reconnect to the session and its associated hardware in future hRun operations.

Note:

The term *session* in the context of hRun is used in two different concepts:

- Design Session – Refers to the entire life-cycle of a design run on hardware, from initial configuration (`setup configure`) to final cleanup (`setup clear`). This includes all activities related to a specific design execution on the HAPS system.

- hRun Script execution – Refers to a single execution of a hRun script, which represent only a part of the overall design session. For example, a script that performs only `setup connect` and `setup disconnect` is considered one hRun script execution within the design session.

Understanding this difference is important when using commands, such as `setup connect`, `setup disconnect`, and `setup clear`, as they manage control and lifecycle across multiple hRun script executions within a single design session.

setup attach

The `setup attach` command allows you to start a secondary session that connects to an already running DUT prototype on HAPS hardware. The hRun process from `setup configure` or `setup connect` remains in control of hardware. This attached session is read-only in terms of hardware control but enables powerful debugging capabilities.

- Used to attach a new session for RTL debugging or design control (signal force, memory access).
- The primary session, created via `setup configure` or `setup connect`, retains full hardware control.
- Only one primary session can exist per setup, but multiple secondary sessions can attach for debugging.

```
setup attach -workdir <workdir_path>
```

`<workdir_path>`: The working directory used in the original `setup configure` command.

- The attached session cannot control the setup by using commands such as `setup clear` and `setup control`. Only connected sessions can control the setup.
- This command is valid only after the setup is complete and the DUT is running.

Exiting a hRun Session

Prerequisite

Before exiting a hRun session, execute one of the following commands:

- `setup clear`: Use this command if the job is finished and hardware can be freed.
- `setup disconnect`: Use this command if the job is not finished. In this case, the design keeps running on hardware and you can reconnect to the session later.

Feedback

After executing one of these commands, execute the `exit` command to exit a hRun session.

Syntax

```
exit [<code>]
```

`[<code>]` is optional. It is used to specify a code that returns a corresponding status (defined for that code in the Tcl script) after exiting.

For example, in the Tcl script, you can define that '0' means that exit is successful without. error.

Alternatively, if the global option `force_clear_on_exit` is set as *on*, the hRun session auto-clears upon exit. If this option is set as *off*, the hRun session auto-disconnects upon exit.

When the global option `force_clear_on_exit` is not set, if you execute the `exit` command without executing the `setup clear` or `setup disconnect` command, the tool does the following:

- **Interactive Mode:** The tool displays a message to execute the `setup clear` or `setup disconnect` command. After you execute one of these commands, the tool exits the hRun session.

5

Managing hRun Jobs

Use the following commands to manage hRun jobs.

Command	Usage
host hrun_scan	To get details of all the hRun jobs running on this host.
host hrun_control	To forcefully terminate a hRun job running on this host.

host hrun_scan

The `host hrun_scan` command allows users to scan jobs on the current host and get details on a given *hrun_id*. With no options, this command scans all hRun jobs on the current host by looking at processes running on host.

Use one of the following options to find the *hrun_id*:

- Execute the `host hrun_scan` command without specifying any options.
- Execute the command specifying *-shs_job_id* to find the *hrun_id* for the given SHS job ID. Every job run using SHS has a *job_id* assigned by the core SLURM. Specify this *job_id* as the value of *-shs_job_id*.

  ```
  host hrun_scan [-shs_job_id <value>]
  ```

After the *hrun_id* is obtained, execute the following command to scan the hRun jobs.

```
host hrun_scan [-format <value>] [-hrun_id <value>]
```

- `-format [TCL|JSON|TABLE]`: Specify the format of the output (Tcl, JSON or Table).
- `-hrun_id <value>`: Specify the value of the *hrun_id* to get more details on the hRun.

host hrun_control

Execute the `host hrun_control` command on the host that has a running hRun session to terminate all jobs on the host. Even if hardware access control is enabled, the hRun session is terminated.

Feedback

Syntax

```
host hrun_control -hrun_id <value>
```

- `-hrun_id <value>`: Specify the ID of hRun to control.

Terminating hRun Jobs

To terminate hRun jobs and free resources, you can use the following options in the order in which they are listed. Proceed to the next option only if the current option does not free resources, as expected.

Option 1: Using the setup_clear command

Execute the `setup clear` command in the active hRun session. If you are unable to use this command, then proceed to the next option.

Option 2: Using the setup_connect command followed by setup_clear command

Execute the `setup connect` specifying the *-workdir* option to connect to the hRun job. See `setup disconnect / setup connect`. If an existing process is connected to the hardware but not accessible, and you want to terminate that process, use the *-admin_action* option to forcefully disconnect the existing process and connect the new process.

Then, execute the `setup clear` command.

Option 3: Using the host commands

Note:

This is an administrative action used to terminate jobs of other users. Perform this action with caution.

If you want to terminate any job of other users, you can execute the following admin commands. Execute the `host hrun_scan` command to get the hRun ID.

Then execute the `host hrun_control` command with the *force_setup_clear* action to clear the jobs on the current host.

Note:

Do not power-cycle HAPS hardware or use Linux commands such as “kill“ to terminate any hRun job, as it can result in locked resources or licenses not getting freed.

6

The Design Prototype

Forcing Signal Values in the Design

The `signal` commands allow you to override values on pre-instrumented signals in the design. This is typically used to set design modes or emulate ASIC fuses during reset sequences.

Note:

Signal forcing and releasing is asynchronous and not cycle-accurate. The exact clock cycle at which the forced value takes effect is not controlled. Therefore, it is recommended to apply signal forces only when the design clocks are stopped.

Some examples of signal commands are:

- signal force
- signal getvalue
- signal release
- signal list

Global Options for Signal Control

- `signal_radix`: Sets the default value format (`binary`, `hex`, `decimal`).
- `signal_name_match_mode`: Controls behavior when a signal name is not found (`silent`, `warn`, `error`).

signal force

The `signal force` command:

- Allows backdoor access to signals without regenerating bitstreams.
- Supports both scalar and vector signals in multiple formats, including decimal, hexadecimal, and binary. The global option signal_radix sets the default signal format.

- Forces only signals instrumented using `$dumpvars` syntax in RTL.
- Temporarily disconnects the signal from its RTL driver and applies the specified value until released using `signal release` command.

Use `signal list` to view available signals.

Syntax

```
signal force [-dict <value>] [-file <filename>] [-name <value>] [-value
 <value>]
```

- `-dict <value>` (optional): Tcl dictionary of signal-value pairs.
- `-file <filename>` (optional): File containing signal-value pairs in the format: `<signal_name> <value>`
- `-name <value>`: Name and bit-width of the signal to force.
- `-value <value>`: Value to apply (binary, decimal, or hexadecimal).

Use `-file` or `-dict` to apply multiple signals in a single transaction.

Examples

```
signal force -name clk -value 1
signal force -dict {clk 1 rst 0}
signal force -file signals.txt
```

---

signal getvalue

The `signal getvalue` command enables reading the value of signals in hexadecimal or binary format.

Syntax

```
signal getvalue [-file <filename>] [-list <signals>] [-name <signal>]
 [-read_value_format <value>]
```

- `-file <filename>`: Provide a file with signal names (newline separated) or the file containing signal-value pairs in the format: `<signal_name> <value>` used as input in the `signal force` command.
- `-list <signals>`: Provide a Tcl list of signal names.
- `-name <signal>`: Specify a single signal to read.
- `-read_value_format <value>`: Set the output format as `hexadecimal`, `binary`, or `decimal`.

Examples

```
signal getvalue -name clk
signal getvalue -list {clk rst}
signal getvalue -file signals_to_read.txt
signal getvalue -list {clk rst} -read_value_format hex
```

Returns

Returns a Tcl dictionary *`TCL_DICT ({signal1 value1} {signal2 value2} …)`* or `TCL_ERROR` on failure.

---

signal release

The `signal release` command stops the external force on the signal value. This allows the signal to be driven by the design itself. After release, the signal returns to normal operation.

Syntax

```
signal release [-file <filename>] [-list <list>] [-name <signal>]
```

- `-file <filename>`: Provide a file with signal names (space or newline separated, supports `#` comments).
- `-list <list>`: Provide a Tcl list of signal names.
- `-name <signal>`: Specify a single signal to release.

Examples

```
signal release -name clk
signal release -list {clk rst}
signal release -file release_signals.txt
```

Returns

Returns `TCL_OK` on success or `TCL_ERROR` on failure.

---

signal list

The `signal list` command generates a list of available dynamic force applied signals in the complete design.

Syntax

```
signal list -file <filename>
```

`-file <filename>`: File path where the output list is saved.

Note:

Signals in `-file` or `-dict` are not applied in a specific order. Use individual commands if ordering is critical.

Managing Memory in the Design

Memory management in hRun enables you to initialize, read, and overwrite memory dynamically during runtime. This capability is essential for:

- Running the design with different memory configurations without recompiling RTL.
- Debugging and verifying memory behavior.
- Updating memory model control registers.

These operations are similar in flow to signal manipulation and support both embedded RTL RAM and external memory models (for example, GDDR5).

Note:

Embedded RAMs must be instrumented during the compile flow to be accessible at runtime. Use the `memory list` command to identify available memories and registers.

Use Cases

- Load test patterns or firmware at design reset.
- Modify memory contents between test iterations.
- Read memory contents for validation or debug.
- Configure memory model registers dynamically.

Supported Memory Types

- Embedded RAMs: Must be pre-instrumented.
- Memory Models: Includes support for control register access.
- memory write
- memory read
- memory list

These commands provide flexibility and efficiency in managing signals and memory during the design and debugging process, enabling dynamic control without requiring re-synthesis or reimplementation.

memory list

The `memory list` command returns a list of available ports for the memory model instance in the design when no switches, arguments or options are provided by `<memory_path>`.

Syntax

```
memory list [-memories] [-registers] [<memory_path>]
```

- `-memories <memory_path>`: Hierarchical memory path of the specified memory model instance.
- `-registers <memory_path>`: Configurable register path of the specified memory model instance.

memory write

The `memory write` command overwrites memory contents during runtime, replacing values originally driven by RTL logic.

Syntax

To write memory from a file to the HAPS system, execute the following command:

```
memory write [-radix (binary|hex)] <memory_path> <filename>
```

- `-radix (binary|hex)`: Specifies the radix format of the input file (`binary` or `hex`). Default: `hex`.
- `<memory_path>`: Path to the memory instance in the design hierarchy.
- `<filename>`: File name to read memory content.

To write a config register of memory model only, execute the following command:

```
memory write_reg <register_path> <value>
```

- `<register_path>`: Path to the memory model register. Use the `memory list -registers` command to retrieve a list of valid register paths.
- `<value>`: Value specified in Verilog-style or hexadecimal format.

memory read

The `memory read` command reads memory content during runtime.

Feedback

Syntax

To read memory from HAPS and store content into a file, execute the following command:

```
memory read [-address <value>] [-size <value>] [-radix (binary|hex)]
 <memory_path> <filename>
```

- `-address <value>`: Start address from which to begin reading the memory.
- `-size <value>`: Number of memory locations to read starting from the default value of the address, which is zero (0).
- `-radix (binary|hex)`: Specifies the radix format of the input file (`binary` or `hex`). Default: `hex`.
- `<memory_path>`: Path to the memory instance in the design hierarchy.
- `<filename>`: Name of the output file where the memory content is saved.

To read a configuration register of the memory model only, execute the following command:

```
memory read_reg <register_path>
```

`<register_path>`: Path to the memory model register. Use the `memory list -registers` command to retrieve a list of valid register paths.

7

Debugging the Design Using hRun

To begin debugging your RTL with hRun, you must first complete the prototyping and instrumentation steps using the HAPS® ProtoCompiler. During this phase, define the signals you want to monitor and generate an Instrumentation Design Constraints (IDC) file. This file includes the list of instrumented signals and breakpoints required for runtime analysis.

Once the design is synthesized and programmed with the debuggable version, export the runtime files. These files serve as input to hRun, which allows you to analyze and debug the design as it runs on the target HAPS hardware.

hRun provides a unified Tcl-based shell for all debugging operations. It also supports attaching a new session to an already running design, enabling collaborative or multi-stage debugging workflows.

A key feature of hRun debugging is the Intelligent In-Circuit Emulator (IICE), which is automatically inserted into the design by ProtoCompiler. IICE enables advanced runtime debugging, including signal sampling, triggering, and data capture during hardware emulation. Through IICE commands, you can configure and control debug behavior with precision, automate data collection, and gain deep insights into system performance.

Configuring the IICE Sample Clock

To configure the IICE sample clock for DTD and read IICE units, when using a Clock Control Module (CCM), execute the following command:

```
iice clock [-edge <positive|negative|both>] [-iice <value>] [-list]
[-select <driver_clock|composite_clock|<cclock path>>] [-stepping
<value>]
```

where:

- `[-edge <positive|negative|both>]`: Specify the sample clock edge. For driver clock and composite clock, you can specify only positive edge.
- `[-iice <value>]`: Specify the name of the IICE unit to configure (includes 'all'). If the name is not specified, the option returns the name of the IICE clock.
- `[-list]`: List all available sample clocks.

- `[-select <driver_clock|composite_clock|<cclock path>>]`: Specify the sample clock. Default: *driver_clock*.
- `[-stepping <value>]`: For Readback IICEs, specify the number of sample clock events between each cycle.
- `-use_controlled_clks <value>`: For DTD buffer type and for readback (GSV), specify the value as *3* to switch dynamically between positive, negative or both edges of the controlled clocks during runtime. With this option, you must instrument the driver clock as sample clock in the IDC.

Tickclock sampling is not supported for BRAM buffer type.

Accessing IICE Information

Changing the current IICE

To change the current IICE, execute the following command:

```
iice current [[<name>]]
```

Checking the IICE status

To check the status of the specified IICE, execute the following command:

```
iice info [-iice <value>] [-list]
```

It returns a table or list of key value pairs of IICE settings.

Figure 1 Example output: iice info

```
>iice info
UI249 IICE information for IICE 'IICE_0'
Parameter          Value
-----------------------------------------------
Sample-clock       cnt_demo_top.clk0@positive
Sample-depth       1000
Sample-buffer      behavioral
Qualified-sampling 0
Always-armed       0
Controller-type    none
Trigger-conditions 4
Trigger-states     4
CEL-file
===============================================
```

Feedback

```
>array set a [iice info -list]
>puts $a(Sample-clock)
cnt_demo_top.clk0@positive
>foreach {k v} [iice info -list] {puts "$k $v"}
Sample-clock cnt_demo_top.clk0@positive
Sample-depth 1000
Sample-buffer behavioral
Qualified-sampling 0
Always-armed 0
Controller-type none
Trigger-conditions 4
Trigger-states 4
CEL-file
```

Checking Instrumentation Information for All Signals

Execute the following command:

```
iice instrumentation [-iice <value>] [-list] [-name <signal path name regexp>]
```

Figure 2 Example Output: iice instrumentation

```
>iice instrumentation
UI250 Instrumented signals for IICE 'IICE_0'
Signal                                    Width Trigger Sample Partition Trigger-setting
------------------------------------------------------------------------------------------
cnt_demo_top.u_cnt_demo_inst1.capture_cnt  16    x       x      mb1.uA
cnt_demo_top.u_cnt_demo_inst1.capture_flag 1     x       x      mb1.uA
cnt_demo_top.u_cnt_demo_inst2.capture_cnt  16    x       x      mb1.uA
cnt_demo_top.u_cnt_demo_inst2.capture_flag 1     x       x      mb1.uA
cnt_demo_top.u_cnt_demo_inst3.capture_cnt  16    x       x      mb1.uB
cnt_demo_top.u_cnt_demo_inst3.capture_flag 1     x       x      mb1.uB
cnt_demo_top.u_cnt_demo_inst4.capture_cnt  16    x       x      mb1.uB
cnt_demo_top.u_cnt_demo_inst4.capture_flag 1     x       x      mb1.uB
==========================================================================================
```

```
>foreach {sname width trigger sample partition tsetting} [iice instrumentation -list] {
puts "$sname $width $trigger $sample $partition $tsetting"
}
cnt_demo_top.u_cnt_demo_inst1.capture_cnt 16 1 1 mb1.uA
cnt_demo_top.u_cnt_demo_inst1.capture_flag 1 1 1 mb1.uA
cnt_demo_top.u_cnt_demo_inst2.capture_cnt 16 1 1 mb1.uA
cnt_demo_top.u_cnt_demo_inst2.capture_flag 1 1 1 mb1.uA
cnt_demo_top.u_cnt_demo_inst3.capture_cnt 16 1 1 mb1.uB
cnt_demo_top.u_cnt_demo_inst3.capture_flag 1 1 1 mb1.uB
cnt_demo_top.u_cnt_demo_inst4.capture_cnt 16 1 1 mb1.uB
cnt_demo_top.u_cnt_demo_inst4.capture_flag 1 1 1 mb1.uB
>
```

Feedback

Listing All IICE Units

Execute the following command to list all existing IICEs within the current scope. Typically, the current scope refers to the entire design.

```
iice list
```

Example output:

```
{BRAMIICE mb1.uA.BRAMIICE DUTIICE Readback}
```

Configuring the IICE Sampler

To configure the IICE sampler, execute the following command:

Syntax

```
iice sampler [-iice <name>] [-qualifiertype <value>] [-sampledepth
 <value>] [-samplemode <mode>] [-triggertime <early|middle|late|0..100%>]
```

- `[-iice <name>]`: Specify the name of the IICE unit to configure. If the name is not specified, the option returns the name of the IICE.
- `[-sampledepth <value>]`: Specify the sample depth.
- `[-samplemode <mode>]`: Specify the IICE sample mode. The mode can be `normal|qualified_fill|qualified_intr|always_armed`.
  - `normal`
  - `qualified_fill`: Specify this mode to sample qualified data until sample buffer is full.
  - `qualified_intr`: Specify this mode to sample qualified data until manually interrupted.
  - `always_armed`: Specify this mode to sample data for the most recent trigger until the next trigger or until it is interrupted.
- `[-triggertime <early|middle|late|0..100%>]`: Specify the trigger time for the run.

Examples

- Set the ring buffer to sample 90% after trigger, 10% before:

  ```
  iice sampler -iice myBramIICE -samplemode normal -triggertime early
  ```

- Sample only when the trigger signal is high until buffer is full:

  ```
  iice sampler -iice myBramIICE -samplemode qualified_fill
  ```

- Sample only when the trigger signal is high:

  ```
  iice sampler -iice myBramIICE -samplemode qualified_intr
  ```

- When the trigger occurs, automatically fill the sample buffer. Repeat until sampling is stopped:

  ```
  iice sampler -iice myBramIICE -samplemode always_armed
  ```

- Set the number of readback cycles for Readback IICE:

  ```
  iice sampler -iice Readback -sampledepth 1000
  ```

- Set the number of samples in the DTD sample depth:

  ```
  iice sampler -iice myDtdIICE -sampledepth 1000000
  ```

Starting and Stopping IICE Runs

The `iice start` command is used to arm the IICE, enabling it to respond to defined trigger conditions. Each IICE instance operates with a specific run behavior and can be cross-triggered by other IICE units through hardware events. This separation from waveform generation allows data capture to span multiple runs, offering greater flexibility in debugging and analysis.

For example, in a setup using Deep Trace Debug (DTD), the DTD IICE is configured to trigger through the `trigger_in` port of the Clock Control Module (CCM). In this scenario, a Readback IICE can be cross-triggered by the DTD IICE, enabling synchronized data capture across multiple IICE units for coordinated debugging.

iice start

Syntax

```
iice start -iice [<iice_name>|all] -timeout <value> -wait
```

- `-iice [<iice_name>|all]`: Specifies the name of the IICE units to run. If not, the current IICE is used.
- `-timeout <value>`: Sets a timeout value to stop the run after the specified number of seconds.
- `-wait`: Enables blocking mode, where the command returns only after the trigger has occurred and all data has been downloaded.

iice stop

Syntax

```
iice stop -iice [iice_name>|all]
```

Stops the specified IICE units for non-blocking use cases and returns immediately.

iice join

Syntax

```
iice join -iice [<iice_name>|all]
```

Switches back to blocking mode for non-blocking use cases, waiting for the completion of the IICE run.

iice status

Syntax

```
iice status -iice [<iice_name>|all]
```

Returns the status of the specified running IICE units for non-blocking use cases

Possible statuses include:

- `WAIT_ON_TRIGGER`: Waiting for a trigger condition.
- `COMPLETED`: Run has finished successfully.
- `{SAMPLING <current cycle>}`: Sampling is in progress, with the current cycle count displayed.
- `DISABLED`: IICE is not active.

Defining IICE Trigger Settings

Use the `iice trigger` command to define the trigger settings.

Note:

Only the CEL trigger setup is supported in hRun.

For more information, see:

- IICE Trigger Commands
- CEL Syntax
- Setting the Target for Sample Data
- Running Deep Trace Debug
- Using BRAM for Debugging
- Using Global State Visibility for Debugging
- Cross Triggering

- Fast Track Debug
- Debug ECO

IICE Trigger Commands

Use the `iice trigger` to specify the CEL trigger settings for BRAM/DTD IICEs.

Syntax

```
iice trigger [-iice <value>] [-cel <filename>]
 -ccm_trigger_enable|-ccm_trigger_disable <trigger in port index>
```

- `<value>`: Specify the IICE instance.
- `<filename>`: Specify the CEL configuration file.

Use the following command to control the Clock Control Module (CCM) trigger ports for GSV/Readback IICE.

```
iice trigger -ccm_trigger_enable|-ccm_trigger_disable <trigger in port index>
```

- `<trigger in port index>`: Specify the port index for which the setting is to be changed. If no port index is provided, the command enables or disables all 32 ports simultaneously.
- If **CCM triggers are disabled** on all ports, the hRun stops the clock directly.
- If **CCM triggers are enabled**, the hRun waits for one enabled trigger to occur before stopping the clock.

Note:

In hRun, you can configure complex counter triggering using the state machine configuration. However, `-counter mode` such as events, cycles, pulse width, and watchdog are not supported.

CEL Syntax

A CEL (Complex Event Language) file is a configuration file used in the HAPS prototyping environment to describe state machines and define triggers for debugging purposes. It is essential for configuring advanced debugging scenarios in HAPS systems.

Variable and Event Declarations

This section outlines the supported syntax and semantics for declaring variables, constants, counters, and events in CEL, used for configuring IICE triggers.

Variable Declarations

Use the `var` keyword to declare signals from the design hierarchy that monitored as trigger conditions for the IICE.

Supported syntax:

```
var var hdl_hierarchy_name;
var var[msb : lsb] hdl_hierarchy_name;
var var[msb : lsb] hdl_hierarchy_name [msb: lsb];
```

- Specify the partial or full range in `hdl_hierarchy_name`.
- Square brackets `[]` are used to specify bit ranges.
- If no range is specified, the variable is treated as a 1-bit signal.
- Variable assignments are not supported.

Constants

Declare unsigned constants using the `constant` keyword. These can be used in event expressions.

```
constant var := unsigned_value
```

Counter definition

Counters are initialized to zero and support only two operations: `reset` and `inc`.

```
counter counter_name;
```

Unary operations

A limited set of unary operators is supported for use with variables. These operators can also be applied to constant values. When a Boolean operation is performed on a variable, it evaluates whether the variable is non-zero

```
event e1 := var;
event e2 := ~var;      // Bitwise NOT
event e3 := !var;      // Logical NOT
event e4 := &var;      // AND reduction
event e5 := ~&var;     // NAND reduction
event e6 := |var;      // OR reduction
event e7 := ~|var;     // NOR reduction
```

Edge-sensitive conditions

The `$rose` and `$fell` tasks are edge-sensitive conditions that apply only to 1-bit variables. In contrast, the `$stable` task supports multi-bit variables and is used to detect whether a signal is remained at a specific value over time.

To check for stability at a particular value, combine `$stable` with an equality condition. There are no restrictions on the value used in this comparison.

Feedback

Example:

To detect a transition from `9'h0FF` to `9'h1FF`, you can use:

```
event event := $rose(var);
event event := $fell(var);
event event := $stable(var);
event event := $rose(var[8]) && var[7:0] == 8'hFF && $stable(var[7:0]);
```

Relational operators

Only comparisons between variables and constants are supported.

```
event event := var == value;

event event := var != value;

event event := var ==? value*;

event event := var !=? value*;
```

The bit value can be `0`, `1`, `x` and `z`. You can also use unary operations (see Unary operations).

Arithmetic operators

For arithmetic operators, only some constant-shift expressions are supported. Variable-shift, addition, subtraction, multiplication, division and modulus are not supported.

```
event event := (var << LiteralConstant1) == LiteralConstant2;

event event := (var >> LiteralConstant1) == LiteralConstant2;

event event := (var <<< LiteralConstant1) == LiteralConstant2;

event event := (var >>> LiteralConstant1) == LiteralConstant2;
```

Concatenation operator

Value concatenation is supported. For variable concatenation, only the following expressions are supported:

```
event event := {var1,var2}

event event := {var1,var2} == value; event event := {var1, var2} !=
 value;

event event := {var1,var2} ==? value*; event event := {var1,var2} !=?
 value*;
```

Repetition operator

Repetition is only valid for constant values; it is not supported for variables.

```
var var := {LiteralConstant {value}};
```

Ternary operator

Ternary operators are allowed in event declarations; an event can be one of these operations:

```
cond_event*? true_event* : false_event*;
```

Parentheses

They are used for these operations: (event), (variable), (operation)

Reuse event

One event can be the condition for another.

In the following example, `evente_2` is equal to `var_1 && var_2 && var_3`.

```
event e_1 := var_1 && var_2; event e_2 :=e_1 && var_3;
```

Notify event

The `keywordnotify` issues a trigger. It can be used with or without state machines.

```
notify<event_name>
```

State Machine Configuration

CEL files allow you to define state machines with states, transitions, and actions.

Example of state machine in CEL

```
fsm fsm_name
{
  initial state 0;
  state state 0
  {
    action_list;
  }
  state state 1
  {
    action_list;
  }
…
}
```

Supported state machine operators:

goto state;	Defines transition relationships between states.

resetcounter;	Resets counter to 0.
inccounter;	Increases counter.
notify;	Issues trigger action.
if-then-else element	*Defines condition for state transition*

Unsupported operators and tasks

- `clock`
- `reset`
- `dec<counter>`
- `$display`
- `$callback`

Conditional Expressions

CEL files support conditional expressions, such as if-then-else to define conditions for state transitions.

- No conditional operators are allowed in an if-then-else statement. For example, these expressions are not supported:

  ```
  if (e0 && e1)or if (!e0)
  ```

- If-then-else element

  ```
  if (event) then action
  if(event) then action_1 else action_2
  ```

- Supported binding methods *occurs* and *sustains*

  For occurs, only the number of times for the event are calculated; for sustains, the event must have continuously occurred.

  ```
  if(event occurs value) then action
  if(event sustains value) then action
  ```

Event Operators

CEL files support logical operators like AND, OR, XOR, and negation for event conditions.

Function	CEL
Negation	`~, !`

Function	CEL
AND operators	`&&, &`
OR operators	`||, |`
XOR operator	`^`
XNOR operator	`~^, ^~`
Boolean constants	`0, 1`

External Triggers

CEL files support the use of external triggers to control state transitions. These triggers allow one IICE unit to respond to external hardware events, even across different time domains.

```
var var_1 titriggerInID;
event e_1 :=var_1;
```

- Declare a variable using the `ti` prefix followed by the external trigger ID (`ti0`).
- Define an event based on the variable's state.
- Use the event in a state machine or with the `notify` keyword to initiate a trigger.

Supported Boolean Conditions:

Only the following operators are supported for external triggers: `!`, `==`, and `!=`.

These can be combined with other logical conditions as needed.

Breakpoint

CEL files can activate breakpoints that are instrumented in the design using the `breakpoints add` command. Multiple breakpoints can be monitored simultaneously.

```
var var_1 breakpoint_name;
event e_1 := var_1;
```

- Declare a variable using the full signal name of the breakpoint.
- Define an event to monitor the breakpoint's activation.
- Use the event in a state machine or with `notify` to trigger actions.

Note:

- Breakpoint names must use Verilog escape identifiers, starting with a backslash (\) and ending with whitespace (space, tab, newline, or form feed).

  For example:`varvar1 top.\if_297.cnt_demo.v:297,43 ;`

- The logical NOT (!) operator cannot be used to deactivate a breakpoint.

Signals From Compiled Netlists (SRS)

CEL files can reference signals from compiled netlists using a specific naming convention. This applies to both standard and unified compile flows.

Instrumented SRS signals must use the following syntax where a new hierarchy name SRS is added in front of the original hierarchy's name.

Standard Compile Flow

```
var var_1
top_module.SRS.srs_hierarchy_name; event e_1 := var_1;
var var_2 [msb : lsb]
top_module.SRS.\hdl_hierarchy_name[msb:lsb] ; event e_2 := var_2;
```

Unified Compile Flow

```
var var_1
SRS.srs_hierarchy_name; event e_1 := var_1;
var var_2 [msb : lsb]
SRS.\hdl_hierarchy_name[msb:lsb] ; event e_2 := var_2;
```

Always use the full SRS signal name as defined during instrumentation to ensure accurate referencing.

CEL Trigger Examples

```
//Trigger at pos edge on signal cnt_demo_top.u_cnt_demo_inst1.hundred:

module cel_1;

var var1 cnt_demo_top.u_cnt_demo_inst1.hundred;

event e0 := $rose(var1);

notify e0;

endmodule


//Trigger at value 200 on signal top.net_cnt_64 and negedge on signal
 top.enable:
```

Feedback

```
module cel_1;

var var1 [7:0] top.net_cnt_64;

var var2 top.enable;

event e0 := var1 == 'd200 && $fell(var2);

notify e0;

endmodule


//Trigger if we see a sequence of value 200 and 40 on one signal followed
 by value 250 on another signal:

module cel_1;

var var1 [7:0] FB1_uA.net_cnt_64;

var var2 [7:0] FB1_uA.inst1.net_cnt;


event e0 := var1 == 'd200;

event e1 := var1 == 'd40;

event e2 := var2 == 'd250;


fsm FSM1 {

    initial S0;

    state S0 {

        if (e0) then goto S1;

    }

    state S1 {

        if (e1) then goto S2;

    }

    state S2 {

        if (e2) then {
```

Feedback

```
            goto S1;
            notify;
        }
    }
}
endmodule

//Trigger once a particular event was seen 334 times:
module cel_2;
var var1       variable_counter.div_by3_end;
var var2       variable_counter.muxed_end;
event e0 := $rose(var1) && var2 == 1'b1;
fsm FSM1 {
    initial S0;
    state S0 {
        if (e0 occurs 334)
        then {
            notify;
        }
    }
}
endmodule
```

Feedback

Setting the Target for Sample Data

To configure waveform generation and signal capture, follow these steps in hRun:

1. Specify the waveform target.
2. Configure the signal selection
3. Control waveform generation using `dump` commands

Note:

hRun currently supports Streaming and Fast Signal Database (FSDB) formats as the primary waveform targets.

For testing, hRun supports a way to validate signal values inside Tcl. The `dump` command is used to append data from multiple runs into the same waveform and it supports stitching waveforms from multiple IICEs into same user waveform.

The `dump` command must be issued before the IICE run. If an IICE does not have an active waveform target, it functions as a trigger-only IICE.

dump

The following `dump` commands provide a structured approach to managing waveform generation and sample data targets, ensuring flexibility and efficiency in debugging and validation workflows.

- Execute the following command to create an FSDB, VCD or ZTDB waveform from one or multiple IICEs.

  ```
  dump {fsdb|vcd|ztdb} [-iice <value>] <filename>
  ```

- Execute the following command to store waveform data in Tcl-readable format.

  ```
  dump text [-iice <value>] <filename>
  ```

  This allows you to source the file within Tcl and access the sample data. This option is recommended only for small data dumps.

- Execute the following command to enable signal selection adjustments using glob patterns for selection and filtering. Initially, this feature is available for readback only, with all signals selected by default.

  ```
  dump signals -select <select_pattern> -filter <filter_pattern> <fid>
  ```

- Execute the following command to disable data dumping, even if new data becomes available during the run `iice` command.

  ```
  dump disable <fid>
  ```

- Execute the following command to re-enable data dumping, if it was previously disabled. By default, data dumping is enabled.

  ```
  dump enable <fid>
  ```

- Execute the following command to close or finalize the specified waveform target.

  ```
  dump close <fid>
  ```

Note:

For designs without CCM, sampling clock frequency in BRAM/DTD IICE FSDB is the same as the actual sampling clock frequency on the hardware.

For designs with CCM, sample clock frequency measurement is disabled since measuring clock frequency for controlled sample clocks may produce inaccurate results. For single FPGA CCM designs in which source and load are in the same FPGA, you can enable the feature using the following command in hRun.

```
iice clock -measure_freq 1
```

Running Deep Trace Debug

To run the Deep Trace Debug (DTD) in hRun, perform the following steps:

1. Set the IICE which must be started.

   ```
   iice current -iice IICE_dtd
   ```

2. Set the DTD sample depth if required.

   ```
   iice sampler -sampledepth <sample depth>
   ```

3. Set the trigger for DTD IICE. Trigger is set using CEL.

   ```
   iice trigger -cel IICE_dtd.cel
   ```

4. Start the IICE.

   ```
   iice start -iice IICE_dtd
   ```

5. Create a new FSDB waveforms for respective IICEs.

   ```
   set f [dump fsdb test_IICE_DTD.fsdb]
   ```

6. Wait for IICE to complete.

   ```
   iice join
   ```

7. Finalize the waveform for respective IICEs.

```
dump close $f
```

Using BRAM for Debugging

To use BRAM for debugging in hRun, perform the following steps:

1. Set the IICE.

```
iice current -iice IICE_bram
```

2. Set the trigger for BRAM IICE.

```
iice trigger -cel IICE_bram.cel
```

3. Start the IICE.

```
iice start -iice IICE_bram
```

4. Create a new FSDB waveforms for respective IICEs.

```
set f [dump fsdb test_IICE_BRAM.fsdb]
```

5. Wait for IICEs to complete.

```
iice join
```

6. Finalize the waveform for IICEs.

```
dump `close $f
```

Using Global State Visibility for Debugging

Global State Visibility (GSV) enables synchronized signal monitoring and debugging using the Readback IICE. The following steps outline how to configure and use GSV with hRun.

1. **Configure Hardware with PLL Disabled**

   Use `setup configure` command to configure the hardware while skipping PLL initialization

   ```
   setup configure -workdir <workdir_path> -designdir <design_directory>
   -hmf <value> -expert_action skip_haps_pll_start
   ```

   - `-expert_action skip_haps_pll_start`: Disables PLL startup during configuration.

   For more information, see setup configure

2. **Enable PLLs After Configuration**

Use `setup configure` command to enable PLLs after configuration.

```
setup control -expert_action reconfigure_plls
```

3. **Configure Readback IICE for GSV**

   Set the current IICE instance:

   ```
   iice current -iice Readback
   ```

   Specify readback parameters:

   ```
   # Set sample depth
   iice sampler -sampledepth 1000

   # Configure sample clock and stepping
   iice clock -select {driver_clock} -stepping 1

   # Define FSDB waveform output
   set f [dump fsdb Readback]
   ```

   In hRun, the LL files of all FPGAs are automatically loaded for Readback. If not required, you can choose the LL file of a specific FPGA using the following command

   ```
   iice current -iice Readback
   iice sampler -fpga mb1.F0
   ```

4. **Configure Trigger Mode**

   Default Mode (CAPIM/Force):

   ```
   iice trigger -ccm_trigger_disable
   ```

   External Trigger Mode:

   ```
   iice trigger -ccm_trigger_enable
   ```

   Enable a specific trigger port (port 1):

   ```
   iice trigger -ccm_trigger_enable -port 1
   ```

5. **Start Readback**

   To begin capturing data:

   ```
   iice start -iice Readback -wait
   ```

6. **Verify Readback Configuration**

   Use the following command to display current Readback IICE settings.

   ```
   iice info
   ```

Feedback

Example output:

```
UI249 IICE information for IICE 'Readback'

Parameter                 Value
-------------------------------------------------
Sample-clock              driver_clock@positive
Sample-depth              1000
Clock-stepping            1
Trigger-enable-port-index 1
```

Examples

CAPIM/Force Readback

```
#Readback settings

iice current -iice Readback

iice sampler -sampledepth 1000

iice clock -select {driver_clock} -stepping 1

set f [dump fsdb Readback]

iice trigger -ccm_trigger_disable

#Start iice

iice start -iice Readback -wait

dump close $f
```

Readback with External BRAM/DTD trigger

```
#Specify BRAM/DTD settings

iice current -iice IICE_DTD

iice trigger -cel wdir/trigger.cel

set f1 [dump fsdb iice_DTD_2.fsdb]

#Readback settings

iice current -iice Readback

iice sampler -sampledepth 1000

iice clock -select {driver_clock} -stepping 1

set f2 [dump fsdb Readback_extn]
```

Feedback

```
iice trigger -ccm_trigger_enable

#Start iice

iice start -iice {IICE_DTD Readback}

iice join -iice {Readback IICE_DTD}

#Stop waveform dump

dump close $f1

dump close $f2
```

Cross Triggering

Cross-triggering enables one IICE unit to initiate a trigger event in another IICE unit, even when they operate in different time domains. This allows for synchronized debugging across multiple instances of IICE.

Cross-Trigger Setup: System to SLP

To configure a cross-trigger from a system-level IICE to an SLP-level IICE, follow the steps below:

1. Configure the SLP IICE (waiting for trigger):

   ```
   iice current -iice FB1_F0.IICE_1

   iice trigger -cel IICE_slp.cel
   ```

2. Configure the system IICE (initiating trigger):

   ```
   iice current -iice IICE_0

   iice trigger -cel IICE_sys.cel
   ```

3. Start both IICE units:

   ```
   iice start -iice FB1_F0.IICE_1

   iice start -iice IICE_0
   ```

4. Create and finalize the waveform:

   ```
   set f [dump fsdb test_IICE_sys_to_slp]

   iice join -iice all

   dump close $f
   ```

Feedback

```
# system to SLP cross trigger
iice current -iice FB1_F0.IICE_1
iice trigger -cel IICE_slp.cel
set f [dump fsdb test_IICE_sys_to_slp]
iice current -iice IICE_0
iice trigger -cel IICE_sys.cel
iice start -iice FB1_F0.IICE_1
iice start -iice IICE_0
iice join -iice all
dump close $f

#SLP to system cross trigger
iice current -iice IICE_0
iice trigger -cel IICE_sys_trig.cel
set f1 [dump fsdb test_IICE_slp_to_sys]
iice current -iice FB1_F0.IICE_1
iice trigger -cel IICE_slp_trig.cel
iice start -iice IICE_0
iice start -iice FB1_F0.IICE_1
iice join -iice all
dump close $f1
```

These procedures ensure accurate synchronization between IICE units, enabling effective cross-domain debugging and waveform correlation.

Fast Track Debug

Fast Track Debug inserts a Deep Trace Debug (DTD) IICE with state-machine-based triggering on each FPGA and enables the cross-triggering infrastructure. In designs that include a Clock Control Module (CCM), the Fast Track Debug IICE can also be configured to perform Readback using an external trigger.

The following steps outline how to configure and execute Fast Track Debug in hRun.

1. **Define the Trigger Condition**

   Create a CEL file that defines the trigger condition using a state machine. The example below triggers when the signal `cnt_Z` equals zero:

   ```
   module cel_1;

   var var1 [15:0] INCR_FILE.demo_inst1.cnt_Z ;

   event e0 := var1 == 'd0;

   fsm FSM1 {

     initial S0;

     state S0 {
   ```

```
    if (e0 occurs 1)
      then {
        notify;
      }
    }
}
Endmodule
```

2. **Run the Fast Track Debug IICE**

   Use the following commands to configure and start the Fast Track Debug session:

   ```
   iice current -iice FB1_F0.ALWAYS_ON_DBG_IICE
   iice trigger -cel trigger1.cel
   set f1 [dump fsdb aod_f0]
   iice start -wait
   dump close $f1
   ```

   This sequence sets the IICE context, applies the trigger condition, initiates waveform dumping, and starts the IICE in blocking mode until the trigger condition is met and data is captured.

3. **Enabling Automatic Cross-Triggering (Optional)**

   To enable automatic cross-triggering between IICE units:

   - In the CEL file for the IICE that is waiting for a trigger, include a condition for the signal `syn_identify_core_m.modflow_ctrigger_out_MTRIG`.
   - Configure the initiating IICE with its own trigger condition.
   - Start both IICE units as described in the Cross-Triggering section.

   For detailed configuration steps, refer to the Cross Triggering section earlier in this guide.

Debug ECO

For signals instrumented using Debug ECO

Feedback

1. Set the trigger condition in CEL as shown in the following example:

```
module cel_1;

var var1 INCR.demo_inst2.flag ;

event e0 := $rose(var1);

notify e0;

endmodule
```

2. Run the IICE as shown in the following example:

```
iice current -iice FB1_F0.IICE_1

iice trigger -cel my.cel

set f [dump fsdb iice1]

iice start -wait

dump close $f
```

8

Appendix

Hardware Mapping File (HMF) Format

The Hardware Mapping File (HMF) is a JSON-formatted configuration file used in HAPS systems to define the mapping between logical module names specified in the TSD (Target System Description) and the actual serial numbers of the HAPS hardware modules to be used during runtime. The serial numbers of HAPS modules are used as physical locations. The HMF file provides the mapping from logical modules to physical modules.

The HMF ensures that the correct hardware modules are selected and matched to the design's TSD configuration during setup. It is required for successful hardware configuration and must include all modules referenced in the TSD, using the exact same names.

Note:

The TSD file is generated during the compile flow using the user-defined TSS. The names in TSD are the same as used by the user in TSS.

Example

If the TSD defines two modules named `FB1` and `FB2`, the corresponding HMF file should look like:

```
{
  "tsdmaphaps": {
    "FB1": { "serial": "X001234" },
    "FB2": { "serial": "X005678" }
  }
}
```

Design Configuration File (DCF) Format

The Design Configuration File (DCF) is a JSON-formatted file that specifies runtime customization. DCF is used only by the `setup configure` command and remains valid throughout the design's lifecycle on hardware.

Feedback

You can maintain multiple DCF files for the same `design dir`, with each file modified to different runtime configurations or use cases. While optional, DCF provides advanced customization during the `setup configure` flow. Specify only the fields you want to modify, making DCF an efficient tool for runtime adjustments.

Key Features of the DCF File

- **Override Commands**

  These commands allow you to override specific settings:

  - `pllClkFreqOverride`: Use either clock source name or alias clock name to override clock frequency settings on board GCLK PLLs. The unit of frequency is KHz. The maximum frequency is based on the value defined in the FDC constraints.

- **Options**

  Advanced options are available for specialized use cases, such as modular flows. For example:

  - `disableRcvTraining`: Disables receiver training on specific links.
  - `usrLocalReset`: Controls FPGA reset values before and after IP training.

Example:

```
{

 // Override PLL CLK frequency defined in TSD
 "pllClkFreqOverride":
  {
   // module name or alias name in TSD
      "mb1.PLL1.CLK1": "1234",
      "FB2.PLL4_4": "20M",
      "dclk2": "54321"
  },
 "options":
  {
   // Num of Threads to parallel FPGA configuration
      "numOfConfigThreads": "",
   // only valid during configurations

   "hstdm":
   {
     // disable HSTDM receiver connector/block/iobank from the report
         "disableRcvTraining":
     {
            "mb1.uf1" : {"connector": ["J1", "J16"]},
            "mb2.uf2" : {"block": [1, 2], "iobank": [69, 71]}
          }
```

Feedback

```
        }
        "mgtdm":
  {
          // disable MGTDM receiver quad from training report
          "disableRcvTraining":
    {
            "mb1.uf1" : {"quad": [221, 223]},
            "mb2.uf2" : {"quad": [232]}
          }
        }
    },
}
"options": {
        "usrLocalReset": {
                "preInfraIPTraining":{
                     "fpgas" : ["mb1.uA", "mb2.uB"],              // List
 of FPGA
                      "value" : 1|0                                 //
 assert high "1" or low "0" for the list of FPGA
                },
                "postInfraIPTraining":{
                     "fpgas" : ["mb1.uA", "mb2.uB"],              // List
 of FPGA
                      "value" : 1|0                                 //
 assert high "1" or low "0" for the list of FPGA
                  }
         }
  }
```

Configuring HW access control

The HMF type is:

- full_module: All modules in HMF are fully allocated to the design. This type can have unused FPGAs, if the whole module is specified in HMF.

The following options can be configured in the DCF for hardware access control.

- `full_module_access_control : <On|Off>`
  - `Off` (default): All users can control the assigned FPGAs.
  - `On` (recommended): Protects all modules in HMF These modules are locked by the job and other processes cannot access them. They can be freed by terminating hRun process using `host` commands.

Feedback

Configuring Global Options in hRun

The `option set` command allows you to configure global runtime options in hRun. These options apply for the current session and are not retained once session ends. The options must be set to non-default values in each session.

To view help for a specific option, use:

```
option set <option_name> -help
```

Supported Global Options

- `hw_access_timeout`: Sets the maximum wait time (in seconds) for accessing HAPS hardware. The tool retries until the timeout is reached. This is useful in shared environments.

  ```
  option set hw_access_timeout [seconds]
  ```

  Default: `20` seconds

- `signal_name_match_mode`: Defines how hRun handles mismatches in signal names across FPGAs during `signal force` operations.

  Usage:

  ```
  option set signal_name_match_mode [mode]
  ```

  - `silent` (default): Ignore mismatches.
  - `warn`: Display a warning.
  - `error`: Halt with an error.

- `signal_radix`: Sets the numeric format (radix) for values returned by `signal getvalue`.

  Usage:

  ```
  option set signal_radix [radix]
  ```

  - `h`: Hexadecimal (default)
  - `b`: Binary
  - `d`: Decimal

- `wave_DE`: Enables or disables waveform data expansion (DE) in FSDB dumps for DTD IICE.

  Usage:

  ```
  option set wave_DE [0|1]
  ```

  - `1`: Enable DE
  - `0`: Disable DE

- `wave_clock`: Controls whether the sample clock is included in waveform dumps.

  Usage:

  ```
  option set wave_clock [0|1]
  ```

  - `1`: Include sample clock
  - `0`: Exclude sample clock

- `wave_cycle`: Displays the cycle counter around the trigger event in the waveform.

  Usage:

  ```
  option set wave_cycle [0|1]
  ```

  - `1`: Show cycle counter
  - `0`: Do not show cycle counter

- `wave_period`: Specifies the waveform period for one clock cycle, in nanoseconds.

  Usage:

  ```
  option set wave_period [ns]
  ```

  - Example: `option set wave_period 10`

- `wave_realtime`: Enables real-time timestamping in waveform dumps based on the Clock Control Module (CCM).

  Usage:

  ```
  option set wave_realtime [mode]
  ```

  - Valid modes: `off`, `auto`, `emulation`, `simulation`

Memory and Signal File Formats

Supported Memory Write File Formats

The `memory write` command in hRun supports multiple input file formats to initialize memory content. These formats provide flexibility for different use cases, such as sequential loading, address-specific writes, and range-based initialization.

1. Sequential Write from a Start Address

   Each line contains data to be written starting from a specified address. Data is written to consecutive addresses.

   ```
   @32'h0 : 1122FFFF
   3344EEDD
   5555CCBB
   ```

2. Range-Based Write with Repeated Data

   A single data value is written across a specified address range.

   ```
   @32'h0,'h3 : 1122FFFF
   @32'h4,'h7 : 77779988
   ```

3. Single-Line Sequential Write

   A single line contains a start address followed by multiple data words. Data is written sequentially from the specified address.

   ```
   @32'h400 : 1122FFFF 3344EEDD 5555CCBB 6666AAAA 77779988 88887766
    99995544 AAAA3344
   ```

4. Implicit Sequential Write from Address 0

   Each line contains one data word. Data is written sequentially starting from address `0x0`.

   ```
   1122FFFF
   3344EEDD
   5555CCBB
   6666AAAA
   77779988
   88887766
   99995544
   AAAA3344
   ```

5. Explicit Address-Data Pairs

   Each line specifies an address and the corresponding data word.

   ```
   0 1122FFFF
   1 1122FFFE
   2 1122FFFC
   ```

Limitations

- Only ASCII text files are supported for memory operations; no compression formats are allowed.
- Some advanced readmem features (such as updating parsing context or handling X/Z values) are not supported.

- For internal memory, only the simple readmem format is supported.
- Does not support compressed or binary file formats for memory operations.